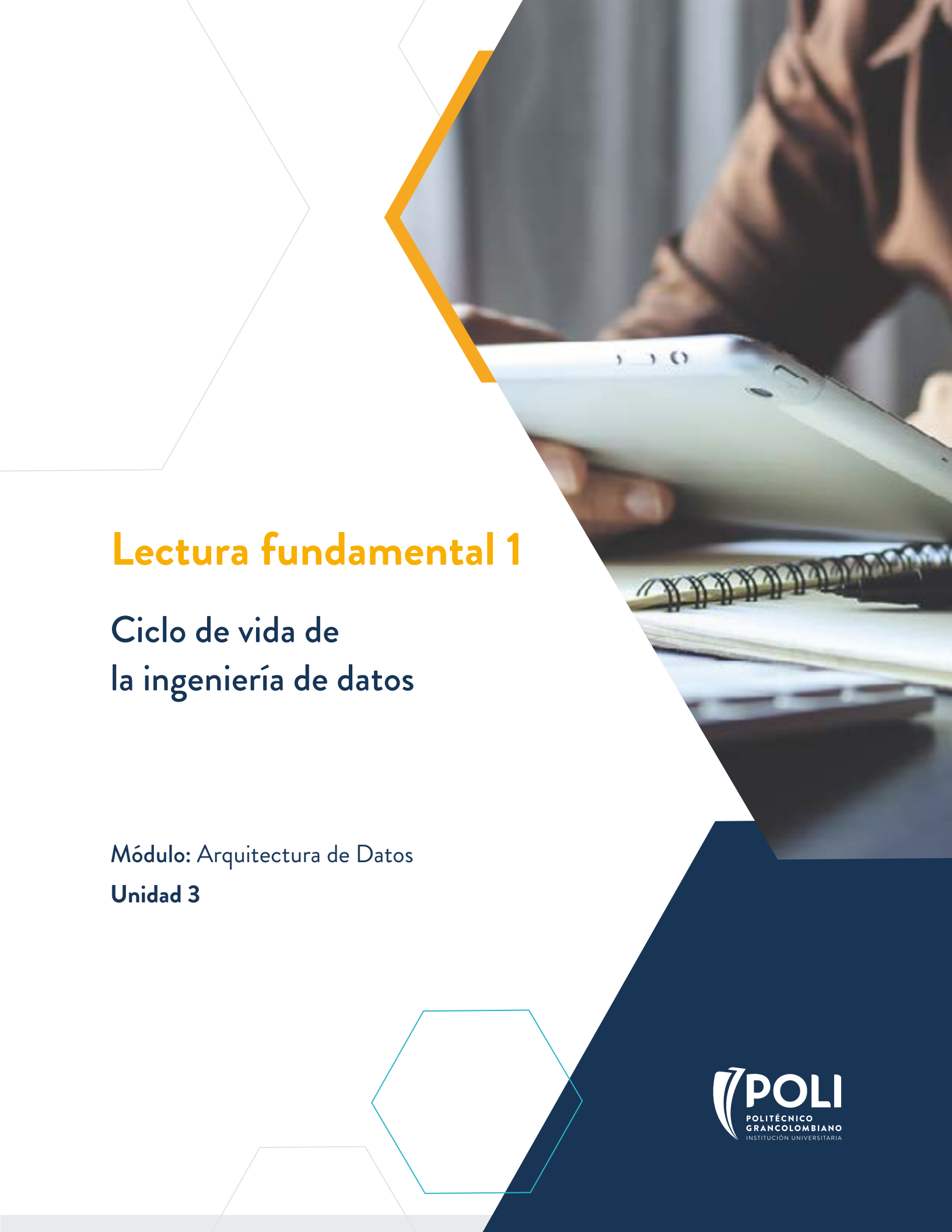




Lectura fundamental 1

Ciclo de vida de la ingeniería de datos

Módulo: Arquitectura de Datos

Unidad 3



Contenido

1

Introducción

2

Fuentes de datos

3

Almacenamiento de datos: Bases de datos relacionales y NoSQL

4

Arquitecturas ACID y BASE

5

Bases de datos en la nube

6

Conclusiones

Palabras clave: base de datos, nube, fuentes de datos, relacional, SQL.

1. Introducción

La arquitectura de datos se construye con base en las fuentes de datos que tengan disponibles las organizaciones, sea que las almacenen dentro de su infraestructura propia, adquieran servicios de almacenamiento en *data centers* o tengan suscripciones a servicios de diferente índole en nube. A partir de esto, se pueden tomar decisiones para proponer una arquitectura de datos adecuada a las necesidades de la organización de acuerdo con los requerimientos actuales y teniendo una visión de posibles requerimientos en un futuro cercano y a largo plazo, siempre y cuando sean proyecciones realistas.

En esta lectura se describen diferentes tipos de fuentes de datos, cómo se almacenan a través de distintas bases de datos, tanto relacionales como las NoSQL. También se aborda el debate sobre los esquemas ACID y BASE, sus ventajas a la hora de proponer esquemas de almacenamiento y los elementos para tener en cuenta cuando se selecciona uno de estos. Finalmente, se muestra el funcionamiento de las bases de datos en nube, que son el tipo de bases de datos hacia el que se están enfocando las organizaciones.

2. Fuentes de datos

Los datos permiten representar hechos y cifras dentro de un contexto específico, sin una estructura, simplemente son una colección desorganizada de elementos. Los datos se pueden crear de forma tanto analógica o digital.

Datos analógicos

Cuando se crean los datos analógicos, son hechos o eventos que ocurren en el mundo real, como el habla vocal, el lenguaje de señas, la escritura en papel o la ejecución de un instrumento musical. Estos datos analógicos suelen ser temporales (Reis y Housley, 2022).

Datos digitales

Los datos digitales son aquellos que se producen a través de la conversión de los datos analógicos a formato digital y también pueden ser el producto original de un sistema digital. Un ejemplo de analógico a digital es una aplicación mensajería móvil que convierte voz analógica a un texto digital, o también conocida como la función de **Dictado**. Un ejemplo de creación de datos digitales son las

transacciones con tarjeta de crédito en plataformas de comercio electrónico. Los clientes realizan pedidos, las transacciones se cargan a sus tarjetas de crédito y la información de las transacciones se almacena en bases de datos. En realidad, los datos están en todas partes del mundo que nos rodea. Cada vez es más frecuente encontrar en nuestro día a día como se capturan datos desde aplicaciones móviles o web, dispositivos IoT con sensores, terminales de tarjetas de crédito, sensores de telescopios, negociación de acciones, etc. (Reis y Housley, 2022).

Para crear arquitecturas de datos y aprovecharlas en un posterior análisis de datos, es crucial conocer los sistemas de las fuentes de datos, dónde están ubicados, qué datos tienen y cómo se generan dichos datos.

2.1. Ejemplos de tipos de sistemas de fuentes de datos

Archivos y datos no estructurados

Son formas de almacenar secuencia de bytes, normalmente se almacenan en repositorios. Son las aplicaciones las que generan estos archivos y pueden almacenar diversa información, como los parámetros locales de una configuración, eventos, registros, imágenes, audio o video.

Esos elementos son importantes porque son los archivos un medio universal de intercambio de datos. Por ejemplo, para el procesamiento de datos de organización, lo más usual es que se descarguen como un archivo de hoja de cálculo o CSV, y es posible que estén en un repositorio o se reciban por correo electrónico (Reis y Housley, 2022).

API (Interfaces de programación de aplicaciones)

Son la forma estándar de intercambiar datos entre sistemas, lo cual simplifica bastante este proceso. Aún muchas APIs presentan cierto nivel de complejidad; incluso con la aparición de diversos servicios y plataformas, así como servicios para automatizar la ingesta de datos, a menudo se necesita invertir bastantes recursos para mantener conexiones API dedicadas.

Bases de datos de aplicaciones (sistemas OLTP)

OLTP es un tipo de base de datos que lee y escribe registros de datos individuales a alta velocidad. También se denominan bases de datos transaccionales, pero esto no significa necesariamente que el sistema en cuestión admita transacciones atómicas. De manera más general, las bases de datos OLTP admiten baja latencia y alta concurrencia. En el caso de los sistemas de gestión de bases de datos relacionales, puede recuperar o actualizar una fila en menos de un milisegundo (ignorando la latencia de la red) y manejar miles de lecturas y escrituras de registros por segundo.



Transacción atómica

Incluye un conjunto de operaciones que son un proceso único que puede ser exitoso o declinado. Si la operación se ejecuta de forma correcta, se ejecuta la operación siguiente.

Al final, las bases de datos OLTP funcionan bien como *backend* de una aplicación, donde miles o incluso millones de usuarios pueden interactuar simultáneamente con la aplicación, actualizando y escribiendo datos al mismo tiempo. En donde los sistemas OLTP no son adecuados, es el caso donde se hacen procesos de análisis a gran escala, donde dada una única consulta debe escanear un gran volumen de datos.

Sistema de procesamiento analítico en línea (OLAP)

Son los sistemas de bases de datos que admiten consultas analíticas interactivas de alta escala y van más allá de los cubos OLAP (matrices de datos multidimensionales). El análisis en línea de OLAP significa que el sistema puede escuchar constantemente consultas, lo que hace que permite que estos sistemas sean adecuados para el análisis interactivo (Reis y Housley, 2022).

A diferencia de los sistemas OLTP, los OLAP están diseñados para realizar grandes consultas analíticas y, por lo general, son ineficientes en el manejo de búsquedas de registros individuales. Por ejemplo, las bases de datos en columnas modernas están optimizadas para escanear grandes cantidades de datos, sin utilizar índices para mejorar la escalabilidad y el rendimiento del escaneo. Cualquier consulta normalmente implica escanear un bloque mínimo de datos, a menudo de 100 MB o más de tamaño.

Entre los sistemas que manejan las organizaciones se necesitan tanto de los OLTP como de los OLAP para cubrir necesidades diferentes en cuanto al almacenamiento y su potencial para generar buenas fuentes de datos para procesos de análisis. Por ejemplo, las bodegas de datos pueden proporcionar datos utilizados para entrenar modelos de aprendizaje automático. Además, un sistema OLAP puede servir un flujo de trabajo ETL inverso, donde los datos obtenidos en un sistema analítico se envían de vuelta a un sistema de origen, como un CRM, una plataforma SaaS o una aplicación transaccional.

LOGs

Este tipo de archivos capturan información sobre eventos que ocurren en el sistema. Por ejemplo, los *logs* que capturan patrones de tráfico y uso en un servidor web. También, en el caso de los sistemas operativos de computadores de escritorio como Windows, macOS y Linux, que necesitan registrar eventos cuando se inicia el sistema y cuando las aplicaciones se inician o fallan. Los *logs* son fuente de datos valiosa para encontrar conocimiento en lo que pasó y cómo prevenir eventos similares en el

futuro. Algunos tipos de logs son: de sistema operativo, aplicaciones, servidores, contenedores, red, dispositivos de IOT (Reis y Housley, 2022). Todos los logs permiten el rastreo de eventos y también de los metadatos de eventos. Como mínimo se puede dar respuesta a estas tres preguntas: ¿quién?, ¿qué? y ¿cuándo?

- **¿Quién?:** persona, sistema o servicio asociado con el evento, por ejemplo, *user-agent* web o ID de usuario.
- **¿Qué pasó?:** evento y metadatos asociados.
- **¿Cuándo?:** la marca de tiempo del evento.

CRUD

CRUD es un patrón ampliamente utilizado en aplicaciones de *software* API's y bases de datos. Por ejemplo, una aplicación web hará un uso intensivo de CRUD para solicitudes HTTP RESTful, así como para su respectivo almacenamiento y posteriormente su recuperación de datos desde la base de datos.

Esta sigla significa crear, leer, actualizar y eliminar, es un patrón transaccional comúnmente utilizado en programación y representa las cuatro operaciones básicas del almacenamiento persistente.

Un principio básico de CRUD es que los datos deben crearse antes de usarse. Una vez creados los datos, se pueden leer y actualizar. Finalmente, es posible que sea necesario destruir los datos.

CRUD garantiza que estas cuatro operaciones ocurrirán con los datos, independientemente de su almacenamiento (Reis y Housley, 2022).

Solo inserción (Insert only)

Este patrón de solo inserción mantiene una entrada de la base de datos de forma directa en la propia tabla, lo que lo hace particularmente útil cuando la aplicación necesita acceso al historial. Por ejemplo, el patrón de solo inserción funcionaría bien para una aplicación bancaria que desee mostrar el historial de direcciones de los clientes. En lugar de actualizar los registros, se insertan registros nuevos con una marca de tiempo que indica cuándo se crearon. Suponga que tiene una tabla con direcciones de clientes. Siguiendo un patrón CRUD, simplemente actualizaría el registro cuando el cliente cambie su dirección. El patrón de solo inserción inserta un nuevo registro de dirección con el mismo ID de cliente. Para leer la dirección del cliente actual utilizando el ID del cliente, necesita

encontrar el último registro con ese ID (Reis y Housley, 2022). Las tablas de aplicaciones CRUD normales suelen utilizar un patrón de análisis independiente de solo inserción. En el patrón ETL de solo inserción, las canalizaciones de datos insertan un nuevo registro en la tabla de análisis de destino cada vez que se produce una actualización de la tabla CRUD.

Mensajes y transmisiones

Cuando se usa una arquitectura basada en eventos, se usan dos términos indistintamente: cola de mensajes y plataforma de transmisión, pero existen diferencias entre los dos. Vale la pena definir y contrastar estos términos porque cubren muchas ideas importantes relacionadas entre los sistemas fuente y las buenas prácticas, así como con tecnologías que abarcan todo el ciclo de vida de la ingeniería de datos (Reis y Housley, 2022).

Un mensaje son datos sin procesar enviados entre dos o más sistemas. Por ejemplo, tenemos el Sistema 1 y el Sistema 2, donde el Sistema 1 envía un mensaje al Sistema 2. Estos sistemas pueden ser diferentes microservicios, un servidor que envía un mensaje a una función sin servidor, etc. Normalmente, un mensaje se envía a través de una cola de mensajes desde un editor a un consumidor y, después de la entrega, se elimina de la cola.

Vale la pena señalar que los sistemas que procesan transmisiones pueden procesar mensajes y las plataformas de transmisión se utilizan con frecuencia para transmitir mensajes. A menudo acumulamos mensajes en flujos cuando queremos realizar análisis de mensajes.

3. Almacenamiento de datos: Bases de datos relacionales y NoSQL

3.1. Bases de datos relacionales

En este tipo de esquemas, se tiene una estructura predefinida donde las tablas se indexan mediante claves primarias, que son campos únicos para cada fila de la tabla. La estrategia de indexación de clave principal está estrechamente relacionada con el diseño de la base de datos en el esquema de almacenamiento físico. Además, las tablas pueden tener varias claves externas: campos con valores que están vinculados a valores de clave principal en otras tablas, lo que facilita las uniones y permite esquemas complejos que distribuyen datos en varias tablas. En particular, es posible diseñar esquemas

estándar. La normalización es una estrategia para garantizar que los datos de registros no se dupliquen en muchos lugares, evitando así la necesidad de actualizar el estado en muchos lugares a la vez y evitando inconsistencias.

Los sistemas RDBMS o de gestión de bases de datos, soportan las características ACID (Date, 2011). La combinación de esquema normalizado, cumplimiento de ACID y soporte para altas tasas de transacciones hace que los sistemas de bases de datos relacionales sean ideales para almacenar estados de aplicaciones que cambian rápidamente. El desafío para los ingenieros de datos es determinar cómo capturar la información de los países a lo largo del tiempo.

Recuerde que...



Características ACID (Date, 2011):

1. **Atomicidad:** conocida como el “todo o nada de una transacción”, para que una transacción termine, se deben haber realizado todas sus partes o ninguna.
2. **Consistencia:** capacidad del sistema para iniciar operaciones que realmente puede concluir.
3. **Integridad:** la ejecución de una transacción no debe afectar a las otras.
4. **Durabilidad:** el conjunto de datos y cambios en una transacción que ya se ejecutó es permanente y no debe haber pérdida de información en el sistema.

3.2. Bases de datos NoSQL

NoSQL significa no solo SQL, término que involucra a toda una clase completa de bases de datos que abandonan el paradigma relacional. Prescinden de varias características de RDBMS, como: fuerte consistencia, conexiones entre tablas y esquemas fijos (Reis y Housley, 2022).



SQL

Lenguaje de consulta estructurado. Es el lenguaje estándar usado para la creación, acceso y actualización de datos en las bases de datos.

Las bases de datos relacionales son excelentes para muchas soluciones de sistemas de información, sin embargo, no son una solución única para toda la organización ni para todo tipo de aplicación de *software*. Revisando la historia de NoSQL, a principios de la década de 2000, las empresas de tecnología como Google y Amazon comenzaron a innovar en sus bases de datos relacionales e introdujeron nuevas bases de datos

distribuidas y no relacionales para escalar sus plataformas web. Hay una gran oferta de bases de datos NoSQL diseñadas para diferentes tipos de uso, a continuación, se describen algunas de las más utilizadas.

Almacenes de valores-clave

Son bases de datos no relacionales que recuperan registros utilizando una clave que identifica de forma única cada registro. Esto es similar a las estructuras de datos de diccionario o mapa *hash* utilizadas en muchos lenguajes de programación, sin embargo, es más escalable (Reis y Housley, 2022).

Los diferentes tipos de bases de datos clave-valor proporcionan un rendimiento diferente, para satisfacer diversas necesidades de aplicaciones. Es frecuente que se usen en el escenario donde se requiera almacenar en caché datos de sesiones en aplicaciones web y móviles que requieren búsquedas ultrarrápidas y altos niveles de simultaneidad. El almacenamiento en estos sistemas suele ser temporal; en caso de cerrarse la base de datos, los datos desaparecen. Estos cachés pueden reducir la carga de la base de datos principal de la aplicación y proporcionar tiempos de respuesta más rápidos (Reis y Housley, 2022). También, los almacenes de valor clave también pueden servir para aplicaciones que requieren alta durabilidad. Es posible que una aplicación de comercio electrónico necesite almacenar y actualizar una gran cantidad de cambios de estado de eventos para un usuario y sus pedidos.

Almacenes de documentos

Es un almacén de valores-clave especializado. En este contexto, un documento es un objeto anidado. Para efectos prácticos se trata un documento como un objeto JSON. Los documentos se almacenan en colecciones y se recuperan mediante clave. Una colección es aproximadamente equivalente a una tabla en una base de datos relacional (Reis y Housley, 2022).

Base de datos orientada a grafos (*graph database*)

Estas bases de datos almacenan explícitamente datos con una estructura de gráfico matemática (con una serie de nodos y aristas). Son una buena opción si se busca analizar la conectividad entre elementos. Se puede pensar en el escenario en el que se requiere de una base de datos de documentos con un documento para cada usuario que describa sus propiedades o configuración.

Se podría agregar un elemento de matriz de conexiones que contenga las identificaciones de los usuarios conectados directamente en un contexto de redes sociales. De esta forma se puede calcular la cantidad de conexiones directas que tiene un usuario y su grado de influencia entre su red. Las bases de datos orientadas a grafos están diseñadas para este tipo de consultas (Reis y Housley, 2022).

Las bases de datos de gráficos admiten modelos de datos enriquecidos tanto para nodos como para bordes. Dependiendo del motor de base de datos de gráficos subyacente, las bases de datos de gráficos utilizan lenguajes de consulta especiales como SPARQL, descripción de recursos.

4. Arquitecturas ACID y BASE

En el contexto de las bases de datos NoSQL, los modelos de consistencia de datos son diferentes de los utilizados en bases de datos relacionales, que puede ser ACID y BASE, cada uno de con sus ventajas y desventajas. A continuación, se describen los dos modelos para que se pueda elegir uno de ellos de acuerdo con las necesidades de negocio que se busque cubrir (Neo4j, 2023).

4.1. Modelo ACID

Las transacciones ACID son propias de las bases de datos relacionales, y como este tipo de bases de datos son las que se han utilizado por más tiempo, se ha considerado el modelo ACID como el estándar. Su característica principal es que genera un entorno seguro en el que trabajar con los datos. La abreviatura ACID significa (Date, 2011; KeepCoding-Team, 2023):

Atómico (Atomic): todas las operaciones necesarias de la transacción se completan con éxito o todas las operaciones se revierten.

Coherencia (Consistent): si se da el caso que una transacción se completa o se revierte, la base de datos siempre está en un estado consistente; no hay actualizaciones parciales ni corrupciones lógicas.

Aisladas (Isolated): cada transacción es independiente de las demás, aunque se ejecuten de forma simultánea. La base de datos gestiona el acceso a las tablas para que las transacciones se completen de forma secuencial.

Duradera (Durable): los resultados de aplicar una transacción son permanentes, incluso si se presentan fallas durante la ejecución. Las bases de datos compatibles con ACID se utilizan ampliamente en sectores como el bancario, la salud y gobierno, donde el almacenamiento de datos tiene una regulación estricta y el volumen de transacciones diario es creciente y bastante alto. Cuando se cumple con modelo ACID, se asegura que, por ejemplo, los clientes de los bancos no tienen que preocuparse de que los saldos de sus cuentas se muestren incorrectamente, porque una base de datos compatible con ACID siempre recuperará los datos más actualizados (Neo4j, 2023).

También, es frecuente ver que pequeñas y medianas empresas utilizan bases de datos compatibles con ACID porque ofrecen estabilidad, su configuración y mantenimiento son conocidos. Debido a la escala relativamente pequeña de datos procesados por este tipo de organizaciones, no hay riesgo en la sobrecarga que el cumplimiento de ACID agrega a las operaciones de la base de datos, y tampoco el tiempo para procesar consultas de reunión (JOIN) que puedan ser complejas y largas.

Las bases de datos más conocidas que permiten la implementación del modelo ACID, incluyen los motores SQL como: Oracle, MySQL, PostgreSQL y MS SQL Server; así como las NoSQL: Neo4j y MongoDB.

La propiedad ACID significa que una vez que se completa una transacción, los datos cumplen con *consistencia de escritura* y son estables en su almacenamiento en disco duro, lo que puede involucrar varias ubicaciones de memoria diferentes. La consistencia de escritura es una ventaja para los desarrolladores de aplicaciones, pero también requiere un bloqueo sofisticado, que suele ser un patrón estricto en un gran número de casos de uso.

Importante

Las bases de datos relacionales aún siguen vigentes y de acuerdo con el tipo de datos y de transacciones que se manejen en el negocio pueden ser la mejor opción como fuente de datos primaria.

Las bases de datos relacionales manejan una serie de restricciones que permiten mantener la integridad de los datos. Estas restricciones son: claves primarias, claves externas, restricciones "No NULL", restricciones "Únicas", restricciones "Predeterminadas" y restricciones "Verificar".

4.2. Modelo BASE

Para muchos contextos de negocio y casos de uso, las transacciones tipo ACID se enfocan mucho en la disminución del riesgo e implementando elementos que aumenten la seguridad de los datos, muchas veces de una forma un poco exagerada con respecto a lo que el negocio y sus necesidades realmente requieren (Neo4j, 2023).

Para el caso de las bases de datos NoSQL, el rendimiento depende de la fragmentación y el escalamiento a gran escala (MPP), gestionar el cumplimiento de ACID es costoso cuando una transacción abarca un buen número de instancias. Como resultado, las bases de datos NoSQL cuyo rendimiento depende en gran medida del escalamiento horizontal suelen utilizar el modelo de transacción BASE (Neo4j, 2023). La abreviatura BASE tiene el siguiente significado:

Accesibilidad básica (*Basic Availability*): la base de datos está disponible la mayor parte del tiempo.

Estado suave (*Soft-state*): los almacenes de datos (*data stores*) no necesitan ser consistentes en escritura y las diferentes réplicas de dichos almacenes no necesitan ser consistentes entre sí todo el tiempo.

Consistencia final (*Eventual consistency*): los almacenes de datos demuestran consistencia en una etapa posterior, no necesariamente en tiempo real, por ejemplo, durante la lectura diferida de los datos. Las bases de datos compatibles con BASE se están usando sobre todo por grandes empresas en contextos donde la regulación es mucho más laxa y que tienen la necesidad de procesar volúmenes de datos bastante grandes de forma diaria. A gran escala, estas empresas están llegando a un punto en el que los costos adicionales de hacer cumplir el modelo ACID genera un sobre costo adicional considerable para la operación. Al final, se busca tener modelos alternativos sin costos adicionales, como lo que se está viendo en muchas bases de datos compatibles con BASE como DynamoDB (AWS, 2023) de Amazon y BigTable (Google Cloud, s. f.) de Google.

También está el caso de organizaciones pequeñas, por ejemplo, una *start-up* que espera que inicialmente no está generando un gran volumen de datos; sin embargo, la cantidad de datos que procesa se estima que pueda crecer en poco tiempo de una forma masiva, y, por tanto, va a necesitar una base de datos compatible con BASE para evitar procesos de migración futuros.

Las bases de datos populares compatibles con BASE incluyen a: BigTable, DynamoDB, Cassandra (Apache Cassandra, s. f.) y Hadoop (Apache Hadoop, s. f.). Lo que propone BASE como modelo de consistencia garantiza una estructura más flexible de lo que formula ACID, y aun así se asegura que los datos puedan guardar consistencia hacia el futuro, sea el caso del momento en el que se leen que corresponde con Riak (Riak, s. f.), o en el otro escenario en que sean consistentes los datos solo en instantes en los que se procesan datos en tiempo pasado como lo que puede ejecutar Datomic (Datomic, s. f.).

5. Bases de datos en la nube

Cuando se habla de bases de datos que son mantenidas en nube, se refiere a los servicios que proponen su creación y gestión a través de una plataforma de computación en se disponga de nube. Se caracterizan en que pueden ejecutar funcionalidades de las bases de datos tradicionales, y adicionalmente tienen mejoras en las posibilidades de configuración y administración con los servicios que disponen los proveedores de nube. Para que sea posible su utilización, primero se debe generar un ambiente donde se instala software en la infraestructura de la nube sobre la cual se pueda implementar la base de datos (IBM, 2021b).

Un escenario frecuente donde se usan estas bases de datos es para el manejo de persistencia de datos de aplicaciones móviles, lo cual es un reto en cuanto a requerimientos no funcionales de escalabilidad y disponibilidad. Para que este esquema funcione adecuadamente, se necesita que las actualizaciones se realicen en una base maestra donde se centralicen los datos. Sin esta configuración se puede generar arquitecturas secuenciales que afectan rendimiento y también van a obstaculizar la ejecución rápida de las aplicaciones (IBM, 2021a; Stedman, 2022).

Dentro de las ventajas de una base de datos en nube, es que posibilitan a las organizaciones proponer un acceso a las bases de datos hasta el extremo más alejado de la red para dispositivos móviles, instalaciones remotas, arquitecturas IoT, sensores y activos que puedan estar habilitados para conectarse a Internet. Este esquema permite asegurar la escalabilidad y permite que las aplicaciones sigan ejecutándose sin conexión.

5.1. ¿Cómo es el funcionamiento de una base de datos en la nube?

Este tipo de sistemas recopilan, entregan, replican y transmiten todos los datos de una organización de forma eficiente y masiva utilizando el concepto de nube pública, privada o híbrida. Algo novedoso de esta forma de trabajo es que las organizaciones ya no necesitan implementar un sistema *middleware* dependiente para realizar consultas de bases de datos. Sin esta limitación, se permite generar consultas desde cualquier parte del mundo y se puedan conectar aplicaciones directamente a estas bases de datos (IBM, 2021).



Middleware

Software y servicios en la nube que brindan servicios y funcionalidades comunes a las aplicaciones, ayudando a los desarrolladores y operadores a crear e implementar aplicaciones de manera más eficiente. El middleware actúa como tejido conectivo entre aplicaciones, datos y usuarios. (Redhat, 2022).

Dentro de las ventajas de los esquemas de bases de datos en nube están la mejora en el rendimiento, el alcance, la movilidad, el tiempo de ejecución de actividad y el ahorro de costos para que las organizaciones puedan aprovechar la flexibilidad que ofrecen los proveedores en nube como (Stedman, 2022):

- Empezar poco a poco e ir aumentando la demanda de servicios a medida que se van utilizando y aprovechando en el almacenamiento y posterior análisis de los datos.
- Escalado elástico bajo demanda, de acuerdo con los nuevos requerimientos del negocio, aumento en productos o servicios, incremento en número de usuarios o usuarios de los datos.
- Distribución de bases de datos en múltiples centros de datos (*data centers*) en diferentes ubicaciones físicas.
- Posibilidad de tener la administración de la nube o permitir que el proveedor de nube la administre.
- Integrar los servicios de proveedores de nube diferentes para aumentar y mejorar la cobertura geográfica, así como acuerdos de nivel de servicio (SLA), costos y cumplimiento con los requerimientos legales que apliquen.

Un ejemplo en el que se ven aplicadas las ventajas mencionadas es el caso de las instituciones financieras que adoptan un concepto híbrido, en el que se usan bases de datos centralizadas, así como la integración con fuentes de datos que no son iguales entre sí y luego son procesados para entregar los datos unificados en formato como puede ser JSON. Al final, estos datos se replican en una base de datos como un servicio y se pueden distribuir en sistemas de datos que estén ubicados en diferentes lugares del mundo.

En la Figura 1 se muestra la situación en la que una organización puede tener diferentes bases de datos ubicadas en diferentes servicios de nube y que al final sean entradas a un sistema de análisis de datos en otra nube.

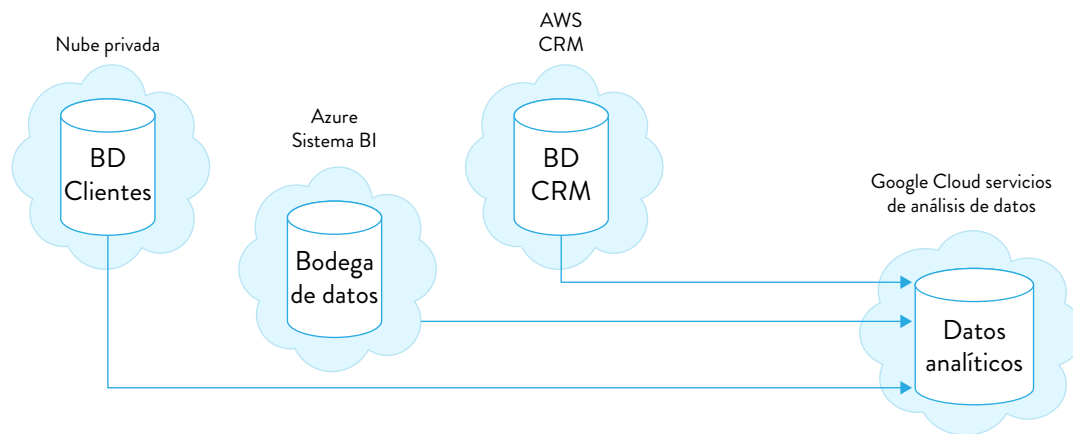


Figura 1. Ejemplo ambiente de base de datos multi-nube

Fuente: elaboración propia

Otra situación que invita a la reflexión sobre las decisiones de arquitectura en los entornos en nube son las aplicaciones móviles. Por ejemplo, si se tienen clientes en Dubái y deben esperar más de tres segundos a que los datos de las aplicaciones móviles se puedan recuperar de las bases de datos en Jamaica, es probable que esos clientes ya no usen esas aplicaciones. La posibilidad de tener la base de datos como un servicio (DBaaS) se puede replicar y distribuir inmediatamente, lo que va a permitir un acceso casi automático que se hace en tiempo real a los datos que pueden estar guardados en cualquier lugar del mundo (IBM, 2021).



DBaaS

Los DBaaS se definen como servicios de bases de datos que son administradas, lo cual posibilita a quienes usan estos servicios, los accesos y usos de los sistemas de base de datos sin que sea necesario pagar por la compra y configuración *hardware* que sea de su propiedad, así como la instalación de su propia base de datos y su respectiva administración tanto del motor como de los datos que se almacenen allí. El proveedor seleccionado para la contratación de los servicios de la nube se responsabiliza de todo el proceso. (IBM, 2021).

Características de una base de datos en la nube

- Es un servicio que tiene todas las características de una base de datos tradicional, con la particularidad que se crea, su acceso y administración es a través de una plataforma que se encuentra disponible en nube.
- Posibilita a los usuarios de organizaciones crear bases de datos en servidores virtuales sin necesidad de adquirir *hardware* o infraestructura dedicada.

- c. Puede ser gestionado por quien contrata el servicio o también adquirir este servicio al proveedor de servicios en nube.
- d. Permite la integración de bases de datos relacionales diferentes como por ejemplo MySQL así como PostgreSQL; también se pueden incluir bases de datos NoSQL como el caso de MongoDB y también Apache CouchDB.
- e. Como se accede es a través de interfaces web y también interfaces de programación de aplicaciones (API) que pueden ser dispuestas por proveedores de servicios en nube.

6. Conclusiones

Para implementar una arquitectura de datos adecuada a la organización, es importante considerar todas las fuentes de datos con las que se cuentan, las bases de datos, sistemas de información, repositorios de archivos, y también sus ubicaciones. Este inventario permite tener todo el panorama de los datos que se están transmitiendo y almacenando en todo el contexto interno o externo del negocio. En el momento de optar por bases de datos en nube, el enfoque más sencillo es utilizar una única plataforma de nube pública. Una opción es implementar bases de datos en una nube híbrida, algunas en la nube pública y otras en una nube privada local.

Referencias

- AWS. (2023). *Amazon DynamoDB*. <https://aws.amazon.com/es/dynamodb/>
- Apache Cassandra. (s. f.). *Open Source NoSQL Database*. https://cassandra.apache.org/_/index.html
- Google Cloud. (s. f.). *Escala tus aplicaciones sensibles a la latencia con la herramienta pionera de NoSQL*. <https://cloud.google.com/bigtable?hl=es>
- Date, C. (2011). *SQL and relational theory: how to write accurate SQL code*. O'Reilly Media, Inc.
- Datomic. (s. f.). *Datomic pro*. <https://www.datomic.com/>
- Apache Hadoop. (s. f.). <https://hadoop.apache.org/>
- IBM. (2021a). *What is a cloud database?* <https://www.ibm.com/topics/cloud-database>
- IBM. (2021b). *What is DBaaS?* <https://www.ibm.com/topics/dbaas>
- KeepCoding-Team. (2023). *¿Qué es el modelo ACID en bases de datos?* KeepCoding Techo School. Recuperado el 11 de abril de 2024. <https://keepcoding.io/blog/que-es-acid-bases-datos/>
- Neo4j. (2023). *Data Consistency Models: ACID vs. BASE Explained*. <https://neo4j.com/blog/acid-vs-base-consistency-models-explained/>
- Redhat. (2022). *What is middleware?* <https://www.redhat.com/en/topics/middleware/what-is-middleware>
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering*. O'Reilly Media, Inc.
- Riak. (s. f.). *The world's most resilient NoSql databases*. <https://riak.com/index.html>
- Stedman, C. (2022). *What is a cloud database? An in-depth cloud DBMS guide*. Techtarget. <https://www.techtarget.com/searchcloudcomputing/definition/cloud-database>



Información técnica

Módulo: Arquitectura de Datos

Unidad 3: Diseños de arquitectura de datos

Autora: Isabel Andrea Mahecha Nieto

Asesora Pedagógica: Doris Amelia Gómez Torres

Diseñador Gráfico: Diego Andrés Calderón

Este material pertenece al Politécnico Grancolombiano.
Prohibida su reproducción total o parcial.